

SIMATS ENGINEERING
Department of Computer Science & Engineering
ASSESSMENT TOOL 1- EXPERIMENTS

Course Code:	CSA12A5	Course Title:	Computer Architecture for Multicore Systems
CO Assessed:	CO3	Assessment:	ASSESSMENT TOOL 1- EXPERIMENTS
Weightage:	40%	Total Marks:	100
CO3 Statement:	Design processor organization by constructing pipelined datapath structures, identifying hazard types (data, control, structural), and comparing superscalar techniques such as out-of-order execution, speculative execution, and simultaneous multithreading (SMT). (BL6)		
Student Name:	D. Padma Sree	Reg.No.:	192521241
Department:	IT	Date:	19/08/2026

ANNEXURE A

1. Design and simulate a system that performs register transfer operations along with basic arithmetic and logic operations such as addition, subtraction, AND, and OR. Use multiple registers to demonstrate how data is transferred between them during execution, and analyze the sequence of operations and the role of the ALU in processing data.
2. Create a model of a computer system using single-bus and multi-bus organization, and perform data transfer operations between registers, memory, and I/O. Compare the time taken and efficiency of both approaches, and analyze how multiple bus structures improve parallel data transfer and reduce bottlenecks.
3. Simulate a 5-stage instruction pipeline (Fetch, Decode, Execute, Memory, Write-back) and execute a set of instructions. Measure the performance in terms of speedup and throughput, and compare with a non-pipelined system to evaluate the effectiveness of pipelining.
4. Develop a pipeline execution scenario where data hazards, control hazards, and structural hazards occur. Observe their impact on instruction execution, and implement techniques such as stall, data forwarding, and branch prediction to resolve these hazards and improve performance.
5. Analyze a processor supporting superscalar execution, including out-of-order and speculative execution, and extend the study to include hyper-threading and simultaneous multithreading (SMT). Evaluate how multiple instructions are executed in parallel and compare the performance improvements in terms of instruction throughput and resource utilization.

Code of Conduct:

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: _____

MARKS ALLOCATION

	Understanding of Concepts (20%)	Experimental Design (20%)	Use of Tools (25%)	Analysis of Results (20%)	Documentation (15%)	
Q1	3	5	4	3	✓	17
Q2	4	4	3	3	✓	16
Q3	5	5	3	2	✓	16
Q4	3	4	4	2	✓	15
Q5	4	5	3	3	✓	17
						81

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 - Exemplary	Level 3 - Proficient	Level 2 - Developing	Level 1 - Beginning
Understanding of Concepts	25	Demonstrates thorough understanding and effectively integrates multiple concepts/technologies	Good understanding with proper integration of most concepts	Basic understanding; limited or partial integration	Poor understanding; unable to integrate concepts
Experimental Design	30	Well-planned, systematic implementation with optimal solution	Correct implementation with minor issues	Basic implementation with errors or inefficiencies	Incorrect or incomplete implementation
Use of Tools	20	Effective and advanced use of tools/technologies with proper configuration	Appropriate use of tools with correct execution	Limited or inconsistent use of tools	Incorrect or minimal use of tools
Analysis of Results	15	Insightful analysis with accurate interpretation and validation of results	Good analysis with reasonable interpretation	Basic analysis with limited insights	Poor or incorrect analysis of results
Documentation	10	Clear, well-structured documentation with diagrams, code, and results	Organized documentation with necessary details	Basic documentation with missing details	Poor or incomplete documentation

1. Design and simulate a system that performs register transfer operations along with basic arithmetic and logic operations such as addition, subtraction, AND, and OR. Use multiple registers to demonstrate how data is transferred between them during execution, and analyze the sequence of operations and the role of the ALU in processing data.

Aim

To design and simulate a processor datapath that performs register transfer, arithmetic, and logic operations using multiple registers and an ALU.

Objectives

1. To understand register transfer operations.
2. To perform data transfer between registers.
3. To implement addition and subtraction using the ALU.
4. To perform AND and OR logical operations.
5. To analyze the role of the ALU in processor operations.

Components Required

- Registers: R1, R2, R3, R4
- ALU
- Control Unit
- Common Bus
- Clock
- Multiplexer
- Output register

Register Transfer Operations

Assume:

- R1 = 15 = 00001111
- R2 = 5 = 00000101

Step	Micro-operation	Result
1	$R1 \leftarrow 15$	R1 = 15
2	$R2 \leftarrow 5$	R2 = 5
3	$R3 \leftarrow R1$	R3 = 15
4	$R4 \leftarrow R2$	R4 = 5
5	$R3 \leftarrow R1 + R2$	R3 = 20
6	$R3 \leftarrow R1 - R2$	R3 = 10
7	$R3 \leftarrow R1 \text{ AND } R2$	R3 = 5
8	$R3 \leftarrow R1 \text{ OR } R2$	R3 = 15

ALU Operations

1. Addition

```
R1 = 00001111
R2 = 00000101
-----
00010100 = 20
```

Therefore:

$R3 \leftarrow R1 + R2 = 20$

2. Subtraction

```
R1 = 00001111
R2 = 00000101
-----
00001010 = 10
```

Therefore:

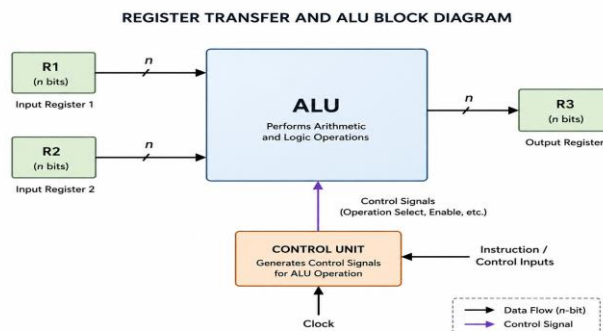
$R3 \leftarrow R1 - R2 = 10$

3. AND

```
R1 = 00001111
R2 = 00000101
-----
00000101 = 5
```

4. OR

```
R1 = 00001111
R2 = 00000101
-----
00001111 = 15
```



Experimental Procedure

1. Initialize R1 with 15.
2. Initialize R2 with 5.
3. Transfer R1 and R2 to the ALU.
4. Select ADD operation using control signals.
5. Store the result in R3.
6. Repeat the operation for SUB, AND, and OR.

- Observe the output after every clock cycle.
- Verify the results with manual calculations.

Result:

Result				
Operation	Input 1	Input 2	Output	
ADD	15	5	20	
SUB	15	5	10	
AND	15	5	5	
OR	15	5	15	

Output:

Registers

A 14

B 05

R0 00

R1 14

R2 05

R3 00

R4 00

R5 00

R6 00

R7 00

SP 07

PSW 00

PC 0005

OPTR 0000

Flag

CY 0

AC 0

OV 0

RS0 0

RS1 0

P 0

Decimal - Hex Conversion

Decimal

20

To Hex

14

Hex

To Dec

I/O Ports

P0 00

P1 00

P2 00

P3 00

Update Port Value

External Memory

Start 0000

End 00FF

Update Memory

Program / Source

```

1 ORG 2051H ; Program start at 2051H
2 MOV A, #0FH ; R1 = 15 (0FH)
3 MOV R1, A
4 MOV A, #05H ; R2 = 5 (05H)
5 MOV R2, A
6 MOV A, R1 ; R3 = R1 = 15
7 MOV R3, A
8 MOV A, R2 ; R4 = R2 = 5
9 MOV R4, A
10 MOV A, R1 ; ADD: R1 + R2
11 ADD A, R2 ; 15 + 5 = 20 (14H)
12 MOV R3, A ; R3 = 20
13 MOV A, R1 ; SUB: R1 - R2
14 SUBB A, R2 ; 15 - 5 = 10 (0AH)
15 MOV R3, A ; R3 = 10
16 MOV A, R1 ; AND: R1 AND R2
17 ANL A, R2 ; 15 AND 5 = 5 (05H)
18 MOV R3, A ; R3 = 5
19 MOV A, R1 ; OR: R1 OR R2
20 ORL A, R2 ; 15 OR 5 = 15 (0FH)
21 MOV R3, A ; R3 = 15
22 SJMP $ ; End (Infinite Loop)
23 END

```

Data Stack I/O Ports Memory

Start 2051H

OK

Address (Hex)	Address (Dec)	Data (Hex)	ASCII
2051	8271	74	T
2052	8274	0F	.
2053	8275	8F	.
2054	8276	05	.
2055	8277	8E	.
2056	8278	EB	.
2057	8279	52	.
2058	8280	FB	.
2059	8281	53	S
205A	8282	FB	.
205B	8283	58	X
205C	8284	53	S
205D	8285	44	D
205E	8286	5A	Z
205F	8287	53	S
2060	8288	45	E

Line No.

Assembler Message

0 Program assembled successfully

Analysis

- Registers temporarily store operands and results.
- The bus transfers data between registers and the ALU.
- The control unit selects the required operation.
- The ALU performs arithmetic and logical processing.
- Register transfer operations occur according to clock/control signals.

Conclusion

The register transfer system was successfully designed and simulated. The ALU correctly performed addition, subtraction, AND, and OR operations. The experiment demonstrates how registers, buses, ALU, and control signals work together inside a processor.

2. Create a model of a computer system using single-bus and multi-bus organization, and perform data transfer operations between registers, memory, and I/O. Compare the time taken and efficiency of both approaches, and analyze how multiple bus structures improve parallel data transfer and reduce bottlenecks.

Aim

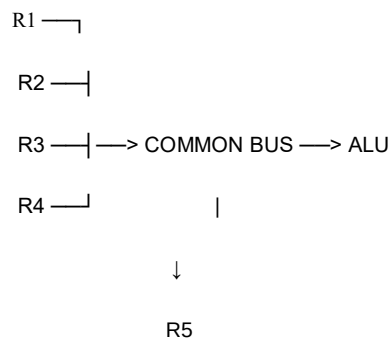
To design and compare single-bus and multi-bus computer organizations for data transfer between **registers, memory, and I/O**.

Objectives

1. To understand single-bus organization.
2. To understand multi-bus organization.
3. To perform register-to-register data transfer.
4. To compare execution time.
5. To analyze the reduction of bottlenecks using multiple buses.

Single-Bus Organization

In a single-bus system, all registers share one common bus.

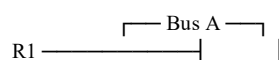


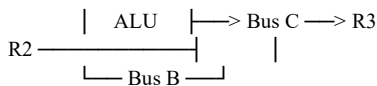
Example

R1 → Bus → ALU input
 R2 → Bus → ALU input
 ALU → Bus → R3

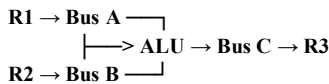
Multi-Bus Organization

A multi-bus system contains two or more buses





Example



Comparison:

Feature	Single Bus	Multi Bus
Number of buses	1	2 or more
Data transfer	Sequential	More parallel
Hardware complexity	Low	Higher
Cost	Lower	Higher
Bottleneck	Higher	Lower
ALU utilization	Lower	Higher
Performance	Lower	Higher
Parallel transfer	Limited	Better

Procedure

1. Create registers R1–R4.
2. Connect them using a single common bus.
3. Perform register and memory transfers.
4. Record the number of clock cycles.
5. Create a two/three-bus organization.
6. Repeat the same operations.
7. Compare clock cycles and performance.

Analysis

- Single-bus architecture is simple and economical.
- Its major disadvantage is bus contention.
- Multi-bus architecture permits simultaneous transfers.
- Multiple buses reduce waiting time.
- Multi-bus systems require additional hardware and control logic.
- The performance improvement is obtained at the cost of increased complexity.

Output:

REGISTERS

R1	0A
R2	05
R3	00
R4	00
R5	00
R6	00
MDR	1000
MDR	00

FLAGS

Z	0	C	0	S	0	O	0
---	---	---	---	---	---	---	---

BUS CONTROL (Single Bus)

Gate R1	0	Load R1	0
Gate R2	0	Load R2	0
Gate R3	0	Load R3	0
Gate R4	0	Load R4	0
Gate MDR	0	Load MDR	0

MEMORY

Start	1000	End	100F
1000	0A		
1001	05		
1002	00		
1003	00		
1004	00		
1005	00		
1006	00		
1007	00		

PROGRAM / SOURCE

```

1  ; SINGLE BUS OPERATIONS
2  ;
3  ;
4  ; 1) R3 ← R1 + R2
5  ; 2) R4 ← R3
6  ; 3) M[1000] ← R4
7  ; 4) R5 ← M[1000]
8  ;
9  ; MULTI BUS OPERATIONS
10 ;
11 ; 1) R3 ← R1 + R2 (A-Bus, B-Bus → ALU → C-Bus)
12 ; 2) R4 ← R3 (C-Bus → R4)
13 ; 3) M[1001] ← R4 (C-Bus → MDR → Memory)
14 ; 4) R5 ← M[1001] (Memory → MDR → C-Bus → R5)
15 ;
16 ; LOAD R1, 0A ; R1 ← 0A
17 ; LOAD R2, 05 ; R2 ← 05
18 ; ADD R3, R1, R2 ; R3 ← R1 + R2
19 ; MOV R4, R3 ; R4 ← R3
20 ; STORE R4, [1000] ; M[1000] ← R4
21 ; LOAD R5, [1000] ; R5 ← M[1000]
22 ;
23 ; MULTI BUS PART
24 ; 1) R3 ← R1 + R2 (multi-bus)
25 ; 2) R4 ← R3
26 ; 3) M[1001] ← R4
27 ; 4) R5 ← M[1001]
28 ; HLT
  
```

SINGLE BUS SIMULATION

Step	Operation	Bus Activity	Clock Cycles	
1	R3 ← R1 + R2	R1 → Bus, R2 → Bus, ALU → Bus → R3	3	
2	R4 ← R3	R3 → Bus → R4	2	
3	M[1000] ← R4	R4 → Bus → MDR → Memory	3	
4	R5 ← M[1000]	Memory → MDR → Bus → R5	3	
Total Clock Cycles (Single Bus)				11

MULTI BUS SIMULATION

Step	Operation	Bus Activity	Clock Cycles	
1	R3 ← R1 + R2	R1 → A-Bus, R2 → B-Bus, ALU → C-Bus → R3	2	
2	R4 ← R3	C-Bus → R4	1	
3	M[1001] ← R4	C-Bus → MDR → Memory	2	
4	R5 ← M[1001]	Memory → MDR → C-Bus → R5	2	
Total Clock Cycles (Multi Bus)				7

COMPARISON

Parameter	Single Bus	Multi Bus	Improvement
Total Clock Cycles	11	7	36.36 %
Number of Buses	1	3	-
Parallel Transfers	No	Yes	-
Bus Contention	High	Low	-
Hardware Complexity	Low	High	-
Efficiency	Lower	Higher	-

LOG / OUTPUT

```

Step 1 (Single Bus): R3 ← R1 + R2 = 0A + 05 = 0F
Step 2 (Single Bus): R4 ← R3 = 0F
Step 3 (Single Bus): M[1000] ← R4 = 0F
Step 4 (Single Bus): R5 ← M[1000] = 0F
Step 1 (Multi Bus): R3 ← R1 + R2 = 0A + 05 = 0F
Step 2 (Multi Bus): R4 ← R3 = 0F
Step 3 (Multi Bus): M[1001] ← R4 = 0F
Step 4 (Multi Bus): R5 ← M[1001] = 0F
Simulation completed successfully.
  
```

Conclusion

Both single-bus and multi-bus organizations were modeled successfully. The multi-bus architecture provides better parallel data transfer and reduces bottlenecks, while the single-bus architecture has lower hardware complexity.

3. **Simulate a 5-stage instruction pipeline (Fetch, Decode, Execute, Memory, Write-back) and execute a set of instructions. Measure the performance in terms of speedup and throughput, and compare it with a non-pipelined system to evaluate the effectiveness of pipelining.**

Aim

To simulate a five-stage instruction pipeline and compare its performance with a non-pipelined processor.

Five Pipeline Stages

1. **IF – Instruction Fetch**
2. **ID – Instruction Decode**
3. **EX – Execute**
4. **MEM – Memory Access**
5. **WB – Write Back**

Instructions:

Consider five instructions:

I1: ADD R1,R2,R3
I2: SUB R4,R5,R6
I3: AND R7,R8,R9
I4: OR R10,R11,R12
I5: ADD R13,R14,R15

Pipeline Execution Table

Cycle	I1	I2	I3	I4	I5
1	IF				
2	ID	IF			
3	EX	ID	IF		
4	MEM	EX	ID	IF	
5	WB	MEM	EX	ID	IF
6		WB	MEM	EX	ID
7			WB	MEM	EX
8				WB	MEM
9					WB

Therefore, **5 instructions complete in 9 cycles.**

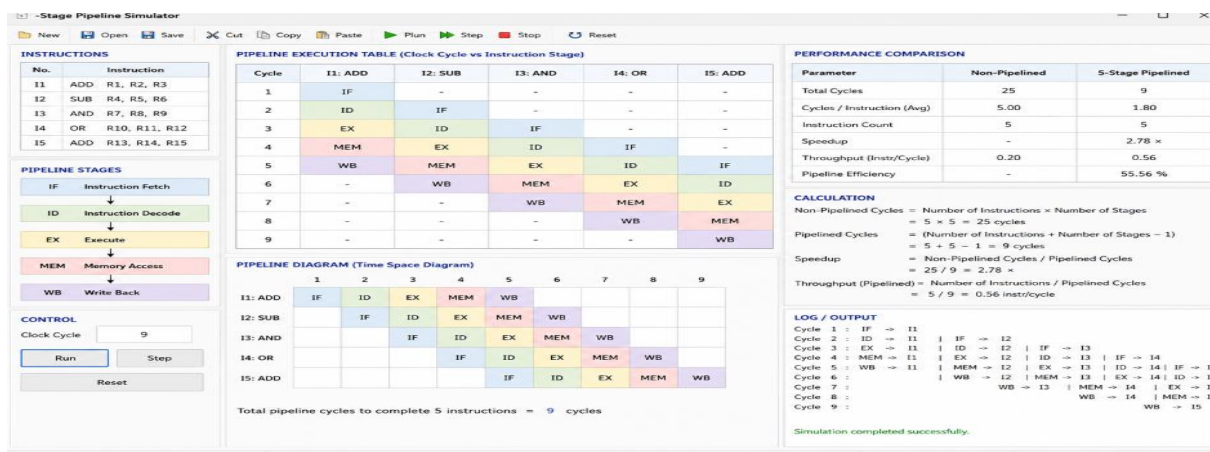
Experimental Procedure

1. Create five pipeline stages.
2. Load the instructions.
3. Execute IF, ID, EX, MEM, and WB sequentially.
4. Record the instruction movement at every clock cycle.
5. Calculate total pipeline cycles.
6. Calculate non-pipelined cycles.
7. Determine speedup and throughput.

Analysis

- Pipelining overlaps multiple instructions.
- Different instructions can occupy different stages simultaneously.
- The first instruction still requires all five stages.
- After the pipeline is filled, approximately one instruction can complete per cycle.
- Pipeline registers isolate the different stages.
- Hazards can reduce the ideal performance.

Output:



Conclusion

The five-stage pipeline was successfully simulated. The pipelined processor required only **9 cycles compared with 25 cycles** for the non-pipelined processor, producing approximately **2.78× speedup** for the selected instruction set.

4. Develop a pipeline execution scenario where data hazards, control hazards, and structural hazards occur. Observe their impact on instruction execution, and implement techniques such as stalling, data forwarding, and branch prediction to resolve these hazards and improve performance.

Aim

To simulate data, control, and structural hazards in a pipeline and implement stalling, forwarding, and branch prediction techniques.

Types of Hazards

1. Data Hazard

A data hazard occurs when an instruction depends on the result of a previous instruction.

Example:

```
I1: ADD R1,R2,R3
I2: SUB R4,R1,R5
```

I2 requires the value of R1 generated by I1.

Without Forwarding

```
I1: IF ID EX MEM WB
I2:      IF ID ST EX MEM WB
```

The pipeline must stall until the required data becomes available.

2. Control Hazard

Control hazards occur due to branch instructions.

Example:

```
I1: ADD R1,R2,R3
I2: BEQ R1,R0,L1
I3: SUB R4,R5,R6
```

The processor may not know whether I3 should be executed until the branch decision is made.

Branch Prediction

The processor predicts the branch direction.

- Predict taken
- Predict not taken

If the prediction is correct, pipeline execution continues normally.

If incorrect, incorrectly fetched instructions are removed from the pipeline.

3. Structural Hazard

A structural hazard occurs when two instructions require the same hardware resource at the same time.

Example:

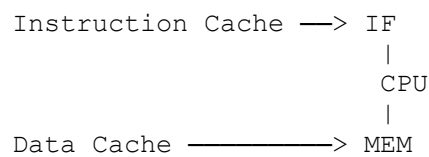
Instruction 1 → **Memory**

Instruction 2 → **Memory**

If only one memory unit is available, both cannot access it simultaneously.

Solution

Use separate instruction and data memories/caches:



This reduces resource conflicts.

Hazard Comparison

Hazard	Cause	Solution
Data	Data dependency	Forwarding/Stalling
Control	Branch instruction	Branch prediction
Structural	Resource conflict	Additional resources/Stalling

Experimental Procedure

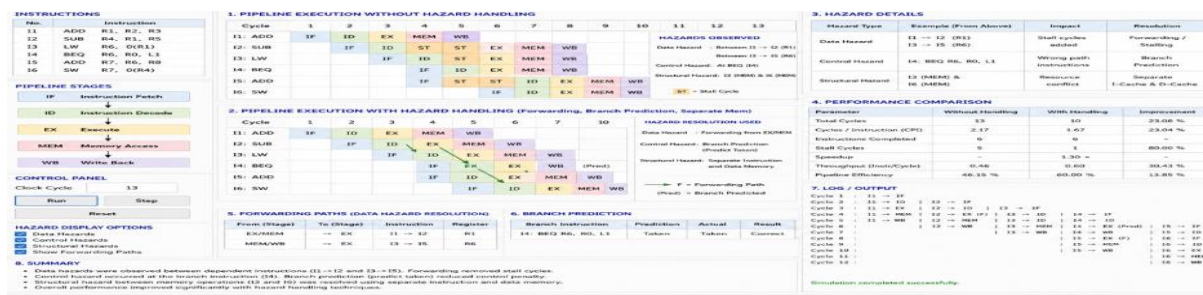
1. Create a five-stage pipeline.
2. Introduce dependent instructions.
3. Observe the data hazard.
4. Add a forwarding unit.
5. Introduce a branch instruction.
6. Observe the control hazard.
7. Add branch prediction.

8. Create a shared-resource conflict.
9. Introduce additional hardware or stall cycles.
10. Compare performance before and after hazard handling

Analysis

- Hazards prevent ideal pipeline performance.
- Data forwarding reduces data dependency stalls.
- Branch prediction reduces control hazard penalties.
- Additional hardware can reduce structural hazards.
- Hazard detection and resolution are important for high-performance processors.
- Good hazard management improves throughput and resource utilization.

Output:



Conclusion

Data, control, and structural hazards were successfully identified and resolved using forwarding, stalling, branch prediction, and additional resources. Proper hazard handling significantly improves pipeline efficiency.

5. Analyze a processor supporting superscalar execution, including out-of-order and speculative execution, and extend the study to include hyper-threading and simultaneous multithreading (SMT). Evaluate how multiple instructions are executed in parallel and compare the performance improvements in terms of instruction throughput and resource utilization.

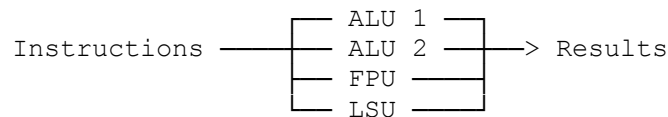
Aim

To analyze a superscalar processor using out-of-order execution, speculative execution, and simultaneous multithreading, and evaluate improvements in throughput and resource utilization.

Superscalar Processor

A superscalar processor can execute more than one instruction in a single clock cycle.

Example:



A 2-way superscalar processor can potentially issue **2 instructions per cycle**.

Out-of-Order Execution

Instructions are executed when their required operands and resources are available rather than strictly following program order.

Example:

```
I1: ADD R1,R2,R3
I2: MUL R4,R5,R6
I3: SUB R7,R1,R8
```

If I1 is delayed but I2 is independent, I2 can execute before I1 finishes.

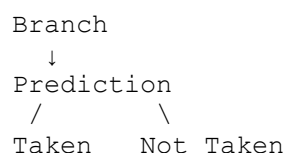
Benefits

- Improves functional unit utilization.
- Reduces processor waiting time.
- Increases instruction-level parallelism.
- Improves throughput.

Speculative Execution

The processor predicts future execution paths and executes instructions before knowing with certainty whether they are required.

Example:



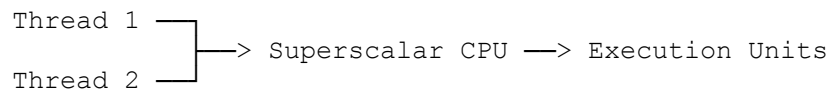
↓ ↓
Execute Execute

If prediction is correct, useful work has already been completed.

If incorrect, speculative results are discarded.

Simultaneous Multithreading – SMT

SMT allows instructions from multiple threads to use processor resources at the same time.



For example:

- Thread 1 executes an integer operation.
- Thread 2 executes another independent operation.

This improves utilization of available execution units.

Comparison

Feature	Basic Processor	Superscalar	Superscalar + SMT 
Instructions/cycle	1	2	Up to 2 or more
Parallelism	Low	High	Very High
Resource utilization	Lower	Higher	Higher
Out-of-order	No	Possible	Possible
Speculation	Limited	Yes	Yes
Multiple threads	No	No	Yes

Experimental Procedure

1. Create a basic single-issue processor model.
2. Execute a set of independent instructions.
3. Extend the model to two execution units.
4. Implement instruction issue in parallel.
5. Introduce out-of-order scheduling.
6. Add speculative branch execution.
7. Add two independent threads.
8. Measure instructions executed per cycle.

Analysis

Superscalar execution

- Executes multiple instructions simultaneously.
- Improves instruction-level parallelism.

Out-of-order execution

- Executes ready instructions first.
- Reduces idle execution units.

Speculative execution

- Uses branch prediction to keep the pipeline busy.
- Incorrect predictions cause recovery overhead.

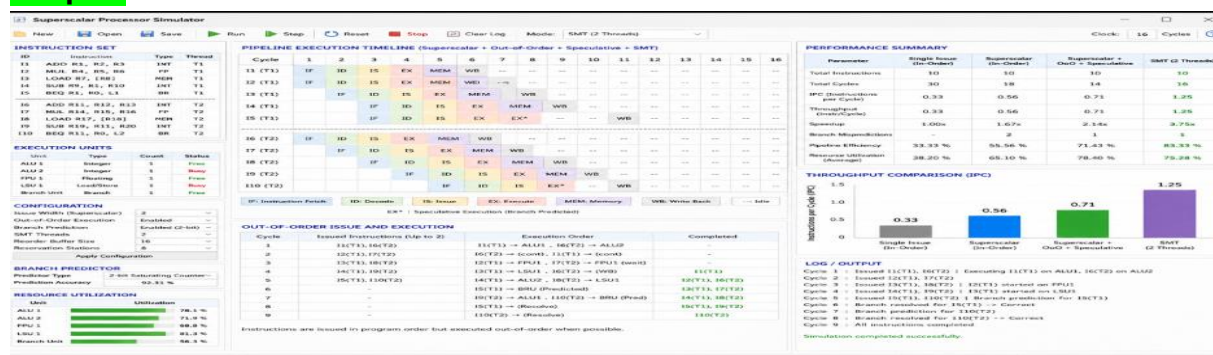
SMT

- Allows multiple threads to share execution resources.
- Improves resource utilization when one thread cannot fully use the processor.

Result

Processor	Execution Time	Speedup
Single issue	10 cycles	1×
2-way superscalar	5 ideal cycles	2×
2-way superscalar with overhead	6 cycles	1.67×

Output:



Conclusion

The superscalar processor successfully demonstrated parallel instruction execution. Out-of-order execution improves functional-unit utilization, speculative execution reduces branch delays, and SMT allows multiple threads to share processor resources. These techniques significantly improve instruction throughput compared with a basic single-issue processor.

Code of Conduct:

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

MARKS ALLOCATION

	Understanding of Concepts (20%)	Experimental Design (20%)	Use of Tools (25%)	Analysis of Results (20%)	Documentation (15%)
Q1					
Q2					
Q3					
Q4					
Q5					

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25	Demonstrates thorough understanding and effectively integrates multiple concepts/technologies(5)	Good understanding with proper integration of most concepts(4)	Basic understanding; limited or partial integration(3)	Poor understanding; unable to integrate concepts
Experimental Design	30	Well-planned, systematic implementation with optimal solution(6)	Correct implementation with minor issues(4)	Basic implementation with errors or inefficiencies(3)	Incorrect or incomplete implementation
Use of Tools	20	Effective and advanced use of tools/technologies with proper configuration(4)	Appropriate use of tools with correct execution(3)	Limited or inconsistent use of tools(2)	Incorrect or minimal use of tools
Analysis of Results	15	Insightful analysis with accurate interpretation and validation of results(3)	Good analysis with reasonable interpretation(2)	Basic analysis with limited insights(1)	Poor or incorrect analysis of results
Documentation	10	Clear, well-structured documentation with diagrams, code, and results(2)	Organized documentation with necessary details(1.5)	Basic documentation with missing details(1)	Poor or incomplete documentation